

TRABALHO FINAL · PROGRAMAÇÃO ORIENTADA A OBJETOS

# GymControl

Sistema de Gestão de Academia

---

**Aluno:** Vinicius Arruda

**Professor:** Rafael Bisogno

**Curso:** Sistemas de Informação — UFN

# Tema e Motivação

---

## O QUE É

Sistema desenvolvido em Java, executado via terminal com menu interativo.

Permite cadastrar alunos e professores, associá-los a planos (mensal, trimestral e anual), registrar pagamentos e remover registros.

Toda a complexidade está na modelagem das classes, aplicando os 7 pilares obrigatórios da POO.

## POR QUE ESCOLHI

- 01 Domínio familiar** fácil de entender e de defender.
- 02 Estrutura natural para POO** pessoas e planos pedem herança e polimorfismo.
- 03 Problemas reais** cadastros inválidos, controle de planos e pagamentos.

# Problema que o Sistema Resolve

---

*Academias de pequeno porte ainda dependem de planilhas e controles manuais, sujeitos a falhas de integridade.*

01

## Cadastros Inválidos

CPFs mal-formatados, nomes vazios e idades irreais corrompem os dados.

*Validação na criação do objeto + bloqueio de CPF duplicado.*

02

## Controle de Planos

Cada plano tem regra própria de cálculo, com descontos diferentes.

*Polimorfismo no `calcularValor()` permite expandir sem reescrever.*

03

## Acesso sem Plano

Sem controle, alunos sem plano ativo acabam usando a academia.

*O sistema identifica alunos sem plano e bloqueia o pagamento.*

# Arquitetura do Sistema

*Quatro pacotes lógicos, separando interface, regras de negócio e dados:*

## Principal

Classe Main, exibe o menu, lê e valida entradas, aciona a Academia.

## models

Entidades do domínio (Pessoa, Aluno, Professor, Plano, Pagamento) e a Academia, que centraliza as regras.

## interfaces

Contrato Exibível, que padroniza a exibição de informações.

## exceptions

Exceção customizada `IdadeInvalidaException`.

# Quem Chama Quem

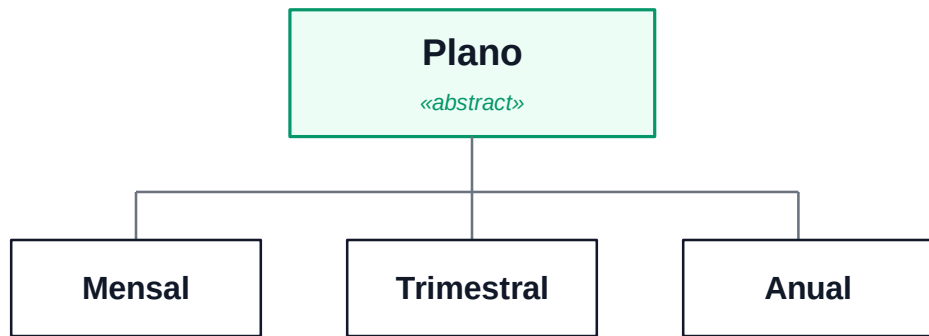
*O usuário interage só com o menu. Cada camada conversa apenas com a seguinte, reduzindo o acoplamento:*



**Exemplo:** ao cadastrar um aluno, a **Main** chama **Academia.adicionarAluno()**, que cria um objeto **Aluno** — e a Pessoa valida os dados na origem.

# Hierarquia de Classes

*Duas hierarquias com relação "É UM":*



**Aluno** é uma **Pessoa** · **PlanoMensal** é um **Plano**

Ambas as superclasses são abstratas e definem métodos que as subclasses implementam.

# Requisitos

---

1

## Classes, Objetos e Atributos

9 classes coesas no pacote models. Objetos criados na Main.

2

## Construtores com Sobrecarga

Aluno(nome, cpf, idade) e Aluno(nome, cpf, idade, plano).

3

## Encapsulamento

Atributos private com getters/setters que validam o estado.

4

## Herança

Pessoa → Aluno, Professor. Plano → Mensal, Trimestral, Anual.

# Requisitos

---

5

## Polimorfismo

Sobrescrita de `exibirDados()` e `calcularValor()`. Polimorfismo de inclusão com `ArrayList<Pessoa>` e `ArrayList<Plano>` na Academia.

6

## Classes Abstratas e Interfaces

Abstratas: `Pessoa` e `Plano` (com método abstrato). Interface `Exibivel` implementada por `Aluno`, `Professor` e `Plano`.

7

## Tratamento de Exceções

Exceção customizada `IdadeInvalidaException`. try-catch na Main trata `IllegalArgumentException` e `InputMismatchException`.



# Decisões de Design

---

**P Por que Pessoa é abstrata e não uma classe comum?**

**R** Pessoa nunca existe sozinha, é sempre aluno ou professor. Abstrata impede instanciá-la e obriga o uso das subclasses.

**P Por que a interface Exibível, se já há herança?**

**R** Plano não é uma Pessoa. A interface é um contrato que atravessa hierarquias diferentes, sem forçar herança.

**P Por que a sobrecarga de construtor em Aluno?**

**R** Permite cadastrar com plano (na hora) ou sem plano (escolhe depois), sem precisar passar null.

# Decisões de Design

---

**P Por que Plano é abstrata se nunca é usada diretamente?**

**R** Ela define o `calcularValor()` como abstrato, obrigando cada subclasse a implementar sua própria regra de desconto.

**P Por que a Academia centraliza tudo em vez de cada classe se gerenciar?**

**R** Concentrar cadastros, buscas e remoções em uma classe reduz o acoplamento. A Main conversa só com a Academia.

**P Por que Aluno e Professor implementam Exibível separadamente?**

**R** Cada um tem sua forma de se exibir. A interface garante o contrato e o polimorfismo trata todos de forma uniforme.

DEMONSTRAÇÃO AO VIVO

# Demo

Sistema em execução no terminal

Link da Demo: <https://youtu.be/YVQb0twPDzk>

Repositório no GitHub: <https://github.com/ViniciusSavianDeArruda/GymControl-POO>

CONSIDERAÇÕES FINAIS

# Obrigado!

Pela atenção!